5.4 TRAVERSING A LINKED LIST

Let LIST be a linked list in memory stored in linear arrays INFO and LINK with START pointing to the first element and NULL indicating the end of LIST. Suppose we want to traverse LIST in order to process each node exactly once. This section presents an algorithm that does so and then uses the algorithm in some applications.

Our traversing algorithm uses a pointer variable PTR which points to the node that is currently being processed. Accordingly, LINK[PTR] points to the next node to be processed. Thus the assignment

PTR := LINK[PTR]

moves the pointer to the next node in the list, as pictured in Fig. 5.8.

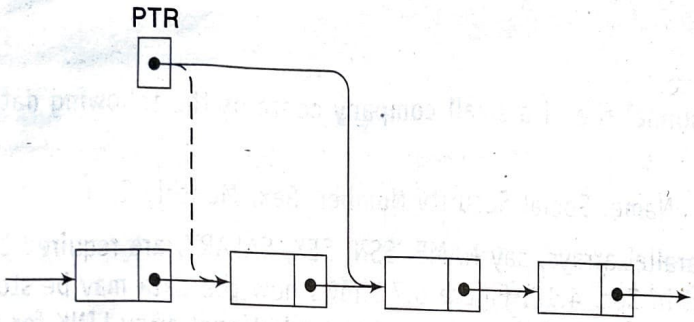


Fig. 5.8 *PTR : = LINK[PTR]*

The details of the algorithm are as follows. Initialize PTR or START. Then process INFO[PTR], the information at the first node. Update PTR by the assignment PTR := LINK[PTR], so that PTR points to the second node. Then process INFO[PTR], the information at the second node. Again update PTR by the assignment PTR := LINK[PTR], and then process INFO[PTR], the information at the third node. And so on. Continue until PTR = NULL, which signals the end of the list.

A formal presentation of the algorithm follows.

Algorithm 5.1: (Traversing a Linked List) Let LIST be a linked list in memory. This algorithm traverses LIST, applying an operation PROCESS to each element of LIST. The variable PTR points to the node currently being processed.

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

Observe the similarity between Algorithm 5.1 and Algorithm 4.1, which traverses a linear array. The similarity comes from the fact that both are linear structures which contain a natural linear ordering of the elements.

Caution: As with linear arrays, the operation PROCESS in Algorithm 5.1 may use certain variables which must be initialized before PROCESS is applied to any of the elements in LIST. Consequently, the algorithm may be preceded by such an initialization step.

5.5 SEARCHING A LINKED LIST

Let LIST be a linked list in memory, stored as in Secs. 5.3 and 5.4. Suppose a specific ITEM of information is given. This section discusses two searching algorithms for finding the location LOC of the node where ITEM first appears in LIST. The first algorithm does not assume that the data in LIST are sorted, whereas the second algorithm does assume that LIST is sorted.

If ITEM is actually a key value and we are searching through a file for the record containing ITEM, then ITEM can appear only once in LIST.

LIST Is Unsorted

Suppose the data in LIST are not necessarily sorted. Then one searches for ITEM in LIST by traversing through the list using a pointer variable PTR and comparing ITEM with the contents INFO[PTR] of each node, one by one, of LIST. Before we update the pointer PTR by

$$PTR := LINK[PTR]$$

we require two tests. First we have to check to see whether we have reached the end of the list; i.e., first we check to see whether

$$PTR = NULL$$

If not, then we check to see whether

$$INFO[PTR] = ITEM$$

The two tests cannot be performed at the same time, since INFO[PTR] is not defined when PTR = NULL. Accordingly, we use the first test to control the execution of a loop, and we let the second test take place inside the loop. The algorithm follows.

Algorithm 5.2 SEARCH(INFO, LINK, START, ITEM, LOC)
LIST is a linked list in memory. This algorithm finds the location LOC of the node where ITEM first appears in LIST, or sets LOC = NULL.

1. Set PTR := START.
2. Repeat Step 3 while PTR ≠ NULL:
3. If ITEM = INFO[PTR], then:
   Set LOC := PTR, and Exit.
   Else:
   Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of If structure.]
   [End of Step 2 loop.]
4. [Search is unsuccessful.] Set LOC := NULL.
5. Exit.

The complexity of this algorithm is the same as that of the linear search algorithm for linear arrays discussed in Sec. 4.7. That is, the worst-case running time is proportional to the number n of elements in LIST, and the average-case running time is approximately proportional to $n/2$ (with the condition that ITEM appears once in LIST but with equal probability in any node of LIST).

Overflow and Underflow

Sometimes new data are to be inserted into a data structure but there is no available space, i.e., the free-storage list is empty. This situation is usually called *overflow*. The programmer may handle overflow by printing the message OVERFLOW. In such a case, the programmer may then modify the program by adding space to the underlying arrays. Observe that overflow will occur with our linked lists when AVAIL = NULL and there is an insertion.

Analogously, the term *underflow* refers to the situation where one wants to delete data from a data structure that is empty. The programmer may handle underflow by printing the message UNDERFLOW. Observe that underflow will occur with our linked lists when START = NULL and there is a deletion.

5.7 INSERTION INTO A LINKED LIST

Let LIST be a linked list with successive nodes A and B, as pictured in Fig. 5.14(a). Suppose a node N is to be inserted into the list between nodes A and B. The schematic diagram of such an insertion appears in Fig. 5.14(b). That is, node A now points to the new node N, and node N points to node B, to which A previously pointed.

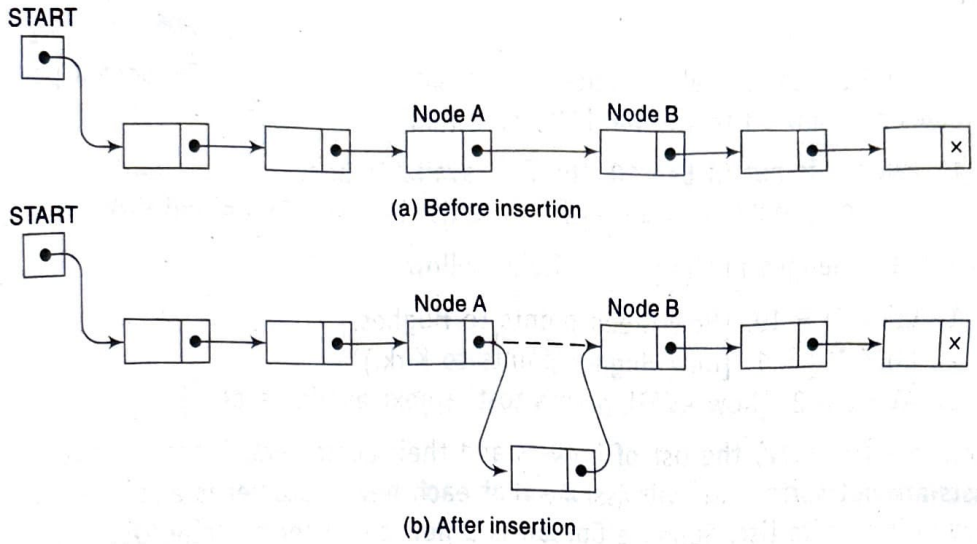


Fig. 5.14

Suppose our linked list is maintained in memory in the form

LIST(INFO, LINK, START, AVAIL)

Figure 5.14 does not take into account that the memory space for the new node N will come from thy AVAIL list. Specifically, for easier processing, the first node in the AVAIL list will be used for the new node N. Thus a more exact schematic diagram of such an insertion is that in Fig. 5.15. Observe that three pointer fields are changed as follows:

(1) The nextpointer field of node A now points to the new node N, to which AVAIL previously pointed.

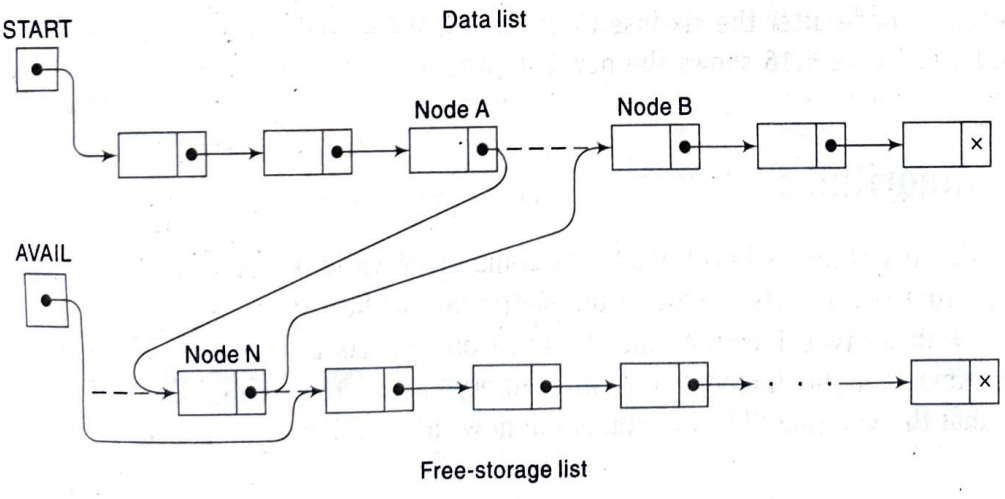


Fig. 5.15

(2) AVAIL now points to the second node in the free pool, to which node N previously pointed.
(3) The nextpointer field of node N now points to node B, to which node A previously pointed.

There are also two special cases. If the new node N is the first node in the list, then START will point to N; and if the new node N is the last node in the list, then N will contain the null pointer.

Example 5.13

(a) Consider Fig. 5.9, the alphabetical list of patients in a ward. Suppose a patient Hughes is admitted to the ward. Observe that

(i) Hughes is put in bed 10, the first available bed.
(ii) Hughes should be inserted into the list between Green and Kirk.

The three changes in the pointer fields follow.

1. LINK[8] = 10. [Now Green points to Hughes.]
2. LINK[10] = 1. [Now Hughes points to Kirk.]
3. AVAIL = 2. [Now AVAIL points to the next available bed.]

(b) Consider Fig. 5.12, the list of brokers and their customers. Since the customer lists are not sorted, we will assume that each new customer is added to the beginning of its list. Suppose Gordan is a new customer of Kelly. Observe that

(i) Gordan is assigned to CUSTOMER[11], the first available node.
(ii) Gordan is inserted before Hunter, the previous first customer of Kelly.

The three changes in the pointer fields follow:

1. POINT[2] = 11. [Now the list begins with Gordan.]
2. LINK[11] = 3. [Now Gordan points to Hunter.]
3. AVAIL = 18. [Now AVAIL points to the next available node.]

(c) Suppose the data elements A, B, C, D, E and F are inserted one after the other into the empty list in Fig. 5.13. Again we assume that each new node is inserted at the beginning of the list. Accordingly, after the six insertions, F will point to E, which points to D, which points to C, which points to B, which points to A; and A will contain the null pointer. Also, AVAIL = 7, the first available node after the six insertions, and START = 6, the location of the first node, F. Figure 5.16 shows the new list (where $n = 10$.)

Insertion Algorithms

Algorithms which insert nodes into linked lists come up in various situations. We discuss three of them here. The first one inserts a node at the beginning of the list, the second one inserts a node after the node with a given location, and the third one inserts a node into a sorted list. All our algorithms assume that the linked list is in memory in the form LIST(INFO, LINK, START, AVAIL) and that the variable ITEM contains the new information to be added to the list.

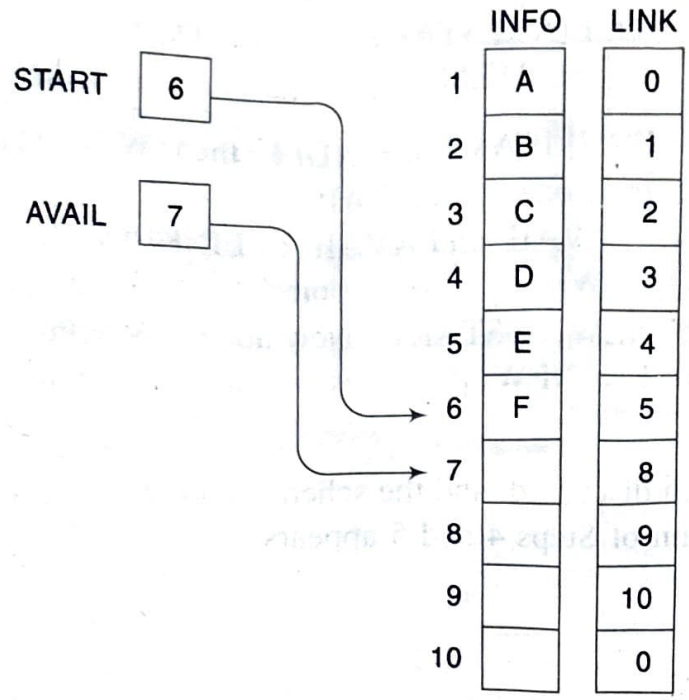


Fig. 5.16

Since our insertion algorithms will use a node in the AVAIL list, all of the algorithms will include the following steps:

(a) Checking to see if space is available in the AVAIL list. If not, that is, if AVAIL = NULL, then the algorithm will print the message OVERFLOW.

(b) Removing the first node from the AVAIL list. Using the variable NEW to keep track of the location of the new node, this step can be implemented by the pair of assignments (in this order)

$$\text{NEW} := \text{AVAIL}, \qquad \text{AVAIL} := \text{LINK[AVAIL]}$$

(c) Copying new information into the new node. In other words,

$$\text{INFO[NEW]} := \text{ITEM}$$

The schematic diagram of the latter two steps is pictured in Fig. 5.17.

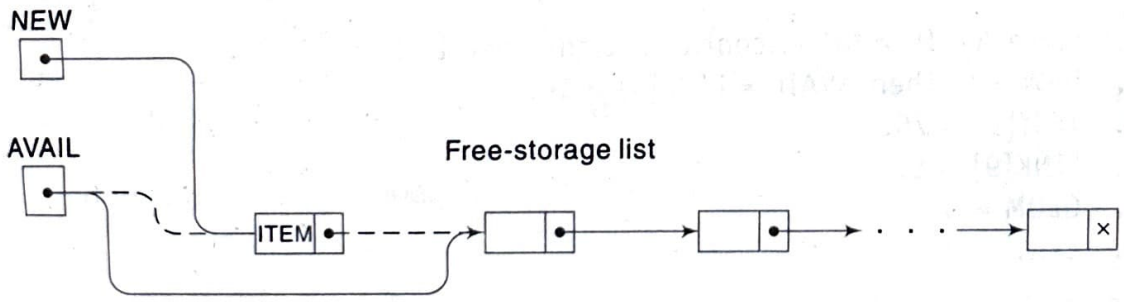


Fig. 5.17

Inserting at the Beginning of a List

Suppose our linked list is not necessarily sorted and there is no reason to insert a new node in any special place in the list. Then the easiest place to insert the node is at the beginning of the list. An algorithm that does so follows.

Algorithm 5.4: INSFIRST(INFO, LINK, START, AVAIL, ITEM)
This algorithm inserts ITEM as the first node in the list.

1. [OVERFLOW?] If AVAIL = NULL, then: Write: OVERFLOW, and Exit.
2. [Remove first node from AVAIL list.]
   Set NEW := AVAIL and AVAIL := LINK[AVAIL].
3. Set INFO[NEW] := ITEM. [Copies new data into new node]
4. Set LINK[NEW] := START. [New node now points to original first node.]
5. Set START := NEW. [Changes START so it points to the new node.]
6. Exit.

Steps 1 to 3 have already been discussed, and the schematic diagram of Steps 2 and 3 appears in Fig. 5.17. The schematic diagram of Steps 4 and 5 appears in Fig. 5.18.

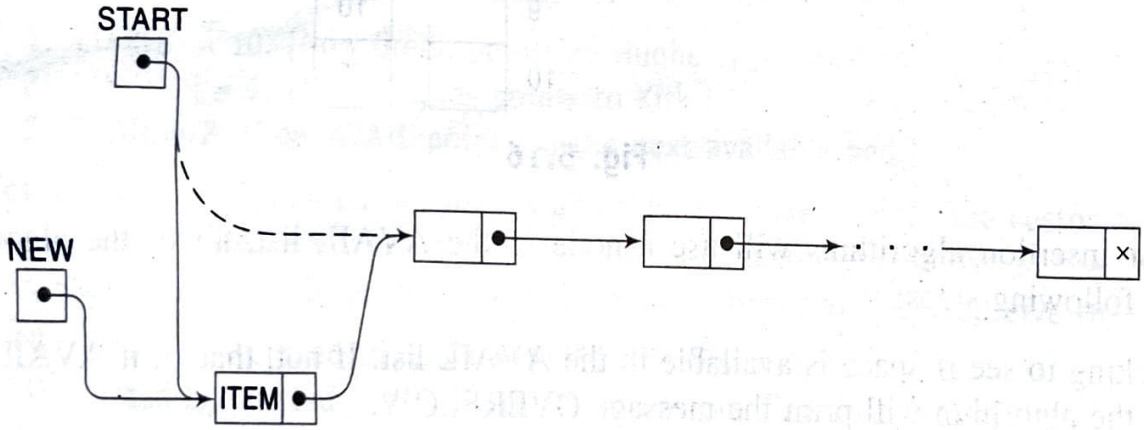


Fig. 5.18 *Insertion at the Beginning of a List*

Example 5.14

Consider the lists of tests in Fig. 5.10. Suppose the test score 75 is to be added to the beginning of the geometry list. We simulate Algorithm 5.4. Observe that ITEM = 75, INFO = TEST and START = GEOM.

INSFIRST(TEST, LINK, GEOM, AVAIL, ITEM)

1. Since AVAIL ≠ NULL, control is transferred to Step 2.
2. NEW = 9, then AVAIL = LINK[9] = 10.
3. TEST[9] = 75.
4. LINK[9] = 5.
5. GEOM = 9.
6. Exit.

Figure 5.19 shows the data structure after 75 is added to the geometry list. Observe that only three pointers are changed, AVAIL, GEOM and LINK[9].

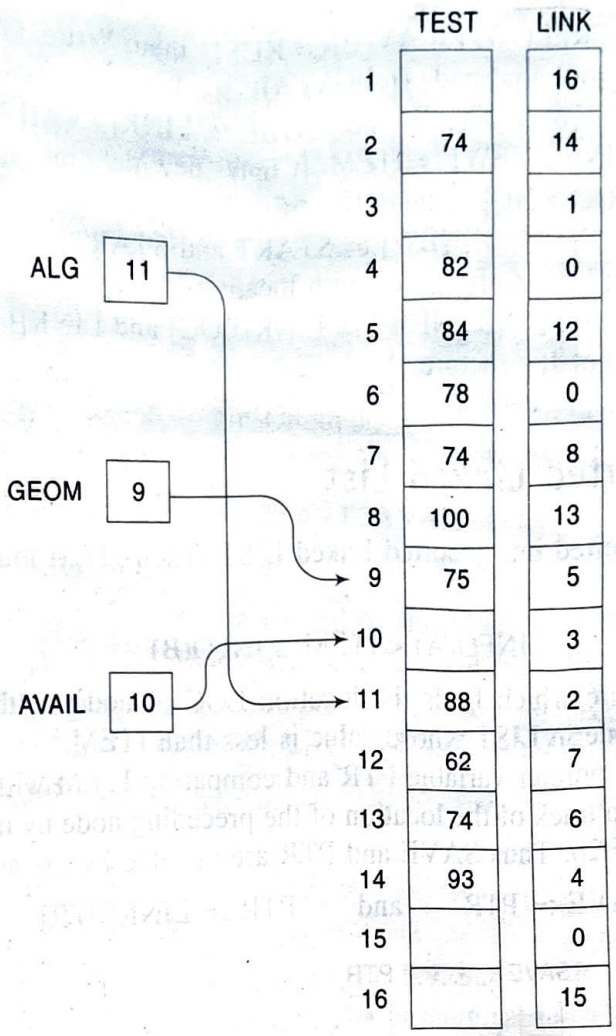


Fig. 5.19

Inserting after a Given Node

Suppose we are given the value of LOC where either LOC is the location of a node A in a linked LIST or LOC = NULL. The following is an algorithm which inserts ITEM into LIST so that ITEM follows node A or, when LOC = NULL, so that ITEM is the first node.

Let N denote the new node (whose location is NEW). If LOC = NULL, then N is inserted as the first node in LIST as in Algorithm 5.4. Otherwise, as pictured in Fig. 5.15, we let node N point to node B (which originally followed node A) by the assignment

LINK[NEW] := LINK[LOC]

and we let node A point to the new node N by the assignment

LINK[LOC] := NEW

A formal statement of the algorithm follows.

Algorithm 5.5: INSLOC(INFO, LINK, START, AVAIL, LOC, ITEM)
This algorithm inserts ITEM so that ITEM follows the node with location LOC or inserts ITEM as the first node when LOC = NULL.

5.8 DELETION FROM A LINKED LIST

Let LIST be a linked list with a node N between nodes A and B, as pictured in Fig. 5.22(a). Suppose node N is to be deleted from the linked list. The schematic diagram of such a deletion appears in Fig. 5.22(b). The deletion occurs as soon as the nextpointer field of node A is changed so that it points to node B. (Accordingly, when performing deletions, one must keep track of the address of the node which immediately precedes the node that is to be deleted.)

Suppose our linked list is maintained in memory in the form

LIST(INFO, LINK, START, AVAIL)

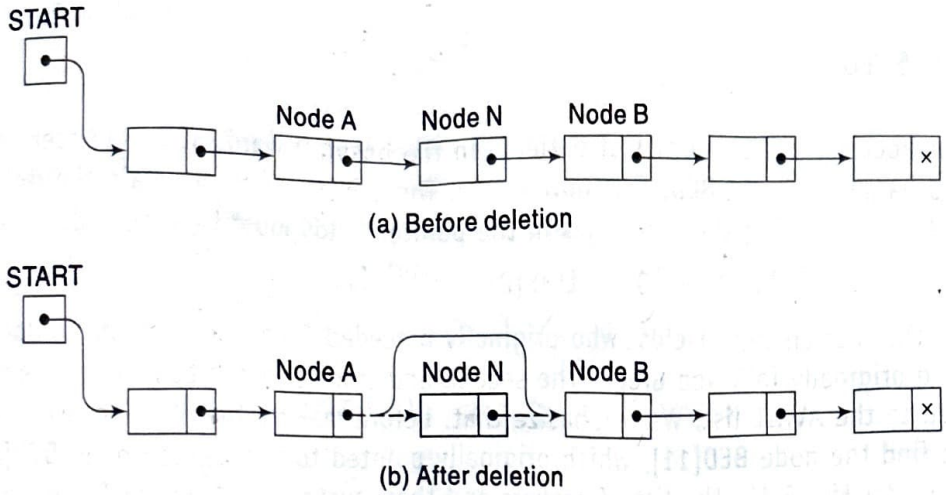


Fig. 5.22

Figure 5.22 does not take into account the fact that, when a node N is deleted from our list, we will immediately return its memory space to the AVAIL list. Specifically, for easier processing, it will be returned to the beginning of the AVAIL list. Thus a more exact schematic diagram of such a deletion is the one in Fig. 5.23. Observe that three pointer fields are changed as follows:

(1) The nextpointer field of node A now points to node B, where node N previously pointed.
(2) The nextpointer field of N now points to the original first node in the free pool, where AVAIL previously pointed.
(3) AVAIL now points to the deleted node N.

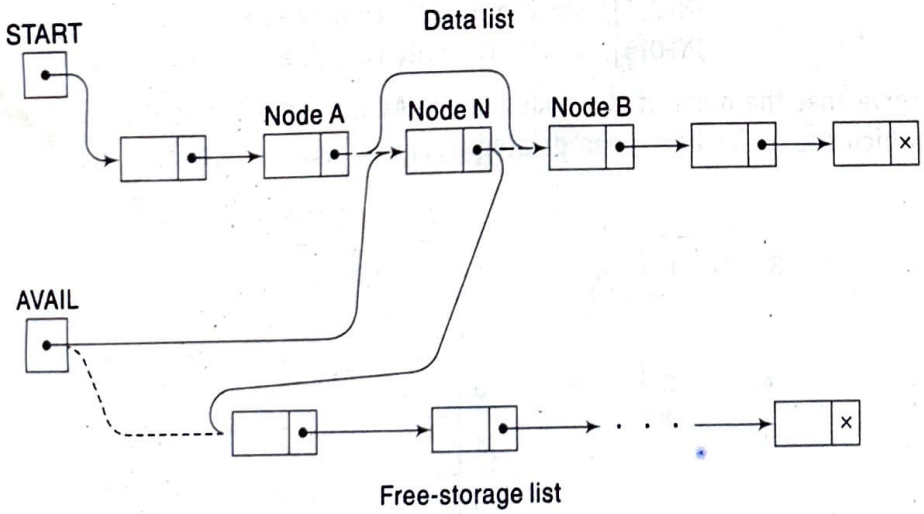


Fig. 5.23

There are also two special cases. If the deleted node N is the first node in the list, then START will point to node B; and if the deleted node N is the last node in the list, then node A will contain the NULL pointer.

Deletion Algorithms

Algorithms which delete nodes from linked lists come up in various situations. We discuss two of them here. The first one deletes the node following a given node, and the second one deletes the node with a given ITEM of information. All our algorithms assume that the linked list is in memory in the form LIST(INFO, LINK, START, AVAIL).

All of our deletion algorithms will return the memory space of the deleted node N to the beginning of the AVAIL list. Accordingly, all of our algorithms will include the following pair of assignments, where LOC is the location of the deleted node N:

LINK[LOC] := AVAIL and then AVAIL := LOC

These two operations are pictured in Fig. 5.25.

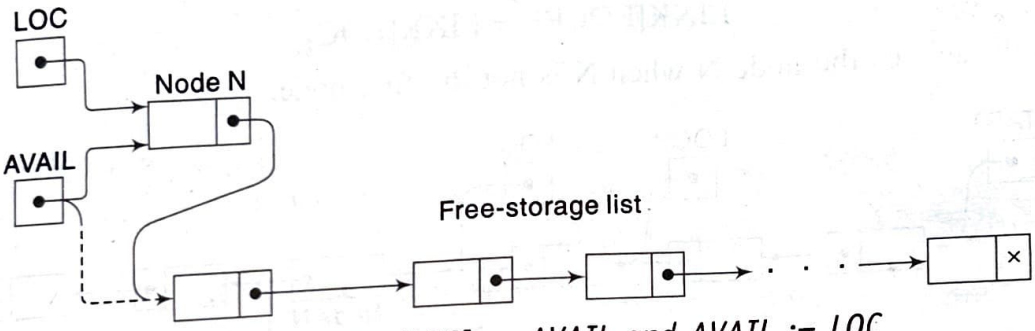


Fig. 5.25 *LINK[LOC] := AVAIL and AVAIL := LOC*

Some of our algorithms may want to delete either the first node or the last node from the list. An algorithm that does so must check to see if there is a node in the list. If not, i.e., if START = NULL, then the algorithm will print the message UNDERFLOW.

Deleting the Node Following a Given Node

Let LIST be a linked list in memory. Suppose we are given the location LOC of a node N in LIST. Furthermore, suppose we are given the location LOCP of the node preceding N or, when N is the first node, we are given LOCP = NULL. The following algorithm deletes N from the list.

Algorithm 5.8: DEL(INFO, LINK, START, AVAIL, LOC, LOCP)
This algorithm deletes the node N with location LOC. LOCP is the location of the node which precedes N or, when N is the first node, LOCP = NULL.

1. If LOCP = NULL, then:
   Set START := LINK[START]. [Deletes first node.]
   Else:
   Set LINK[LOCP] := LINK[LOC]. [Deletes node N.]
   [End of If structure.]
2. [Return deleted node to the AVAIL list.]
   Set LINK[LOC] := AVAIL and AVAIL := LOC.
3. Exit.

Figure 5.26 is the schematic diagram of the assignment

START := LINK[START]

which effectively deletes the first node from the list. This covers the case when N is the first node.

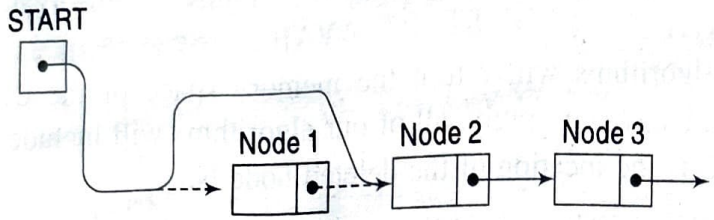


Fig. 5.26 *START := LINK[START]*

Figure 5.27 is the schematic diagram of the assignment

LINK[LOCP] := LINK[LOC]

which effectively deletes the node N when N is not the first node.

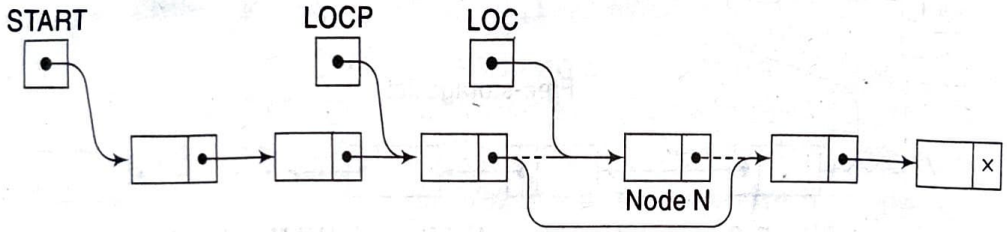


Fig. 5.27 *LINK[LOCP] := LINK[LOC]*

The simplicity of the algorithm comes from the fact that we are already given the location LOCP of the node which precedes node N. In many applications, we must first find LOCP.

Deleting the Node with a Given ITEM of Information

Let LIST be a linked list in memory. Suppose we are given an ITEM of information and we want to delete from the LIST the first node N which contains ITEM. (If ITEM is a key value, then only one node can contain ITEM.) Recall that before we can delete N from the list, we need to know the location of the node preceding N. Accordingly, first we give a procedure which finds the location LOC of the node N containing ITEM and the location LOCP of the node preceding node N. If N is the first node, we set LOCP = NULL, and if ITEM does not appear in LIST, we set LOC = NULL. (This procedure is similar to Procedure 5.6.)

Traverse the list, using a pointer variable PTR and comparing ITEM with INFO[PTR] at each node. While traversing, keep track of the location of the preceding node by using a pointer variable SAVE, as pictured in Fig. 5.20. Thus SAVE and PTR are updated by the assignments

SAVE := PTR and PTR := LINK[PTR]

The traversing continues as long as INFO[PTR] ≠ ITEM, or in other words, the traversing stops as soon as ITEM = INFO[PTR]. Then PTR contains the location LOC of node N and SAVE contains the location LOCP of the node preceding N.

The formal statement of our procedure follows. The cases where the list is empty or where INFO[START] = ITEM (i.e., where node N is the first node) are treated separately, since they do not involve the variable SAVE.

Procedure 5.9: FINDB(INFO, LINK, START, ITEM, LOC, LOCP)
This procedure finds the location LOC of the first node N which contains ITEM and the location LOCP of the node preceding N. If ITEM does not appear in the list, then the procedure sets LOC = NULL; and if ITEM appears in the first node, then it sets LOCP = NULL.

1. [List empty?] If START = NULL, then:
   Set LOC := NULL and LOCP := NULL, and Return.
   [End of If structure.]
2. [ITEM in first node?] If INFO[START] = ITEM, then:
   Set LOC := START and LOCP = NULL, and Return.
   [End of If structure.]
3. Set SAVE := START and PTR := LINK[START]. [Initializes pointers.]
4. Repeat Steps 5 and 6 while PTR ≠ NULL.
5. If INFO[PTR] = ITEM, then:
   Set LOC := PTR and LOCP := SAVE, and Return.
   [End of If structure.]
6. Set SAVE := PTR and PTR := LINK[PTR]. [Updates pointers.]
   [End of Step 4 loop.]
7. Set LOC := NULL. [Search unsuccessful.]
8. Return.

Now we can easily present an algorithm to delete the first node N from a linked list which contains a given ITEM of information. The simplicity of the algorithm comes from the fact that the task of finding the location of N and the location of its preceding node has already been done in Procedure 5.9.

Algorithm 5.10: DELETE(INFO, LINK, START, AVAIL, ITEM)
This algorithm deletes from a linked list the first node N which contains the given ITEM of information.

1. [Use Procedure 5.9 to find the location of N and its preceding node.]
   Call FINDB(INFO, LINK, START, ITEM, LOC, LOCP)
2. If LOC = NULL, then: Write: ITEM not in list, and Exit.
3. [Delete node.]
   If LOCP = NULL, then:
   Set START := LINK[START]. [Deletes first node.]
   Else:
   Set LINK[LOCP] := LINK[LOC].
   [End of If structure.]
4. [Return deleted node to the AVAIL list.]
   Set LINK[LOC] := AVAIL and AVAIL := LOC.
5. Exit.

Remark: The reader may have noticed that Steps 3 and 4 in Algorithm 5.10 already appear in Algorithm 5.8. In other words, we could replace the steps by the following Call statement:

Call DEL(INFO, LINK, START, AVAIL, LOC, LOCP)

This would conform to the usual programming style of modularity.

Example 5.17

Consider the list of patients in Fig. 5.21. Suppose the patient Green is discharged. We simulate Procedure 5.9 to find the location LOC of Green and the location LOCP of the patient preceding Green. Then we simulate Algorithm 5.10 to delete Green from the list. Here ITEM = Green, INFO = BED, START = 5 and AVAIL = 2.

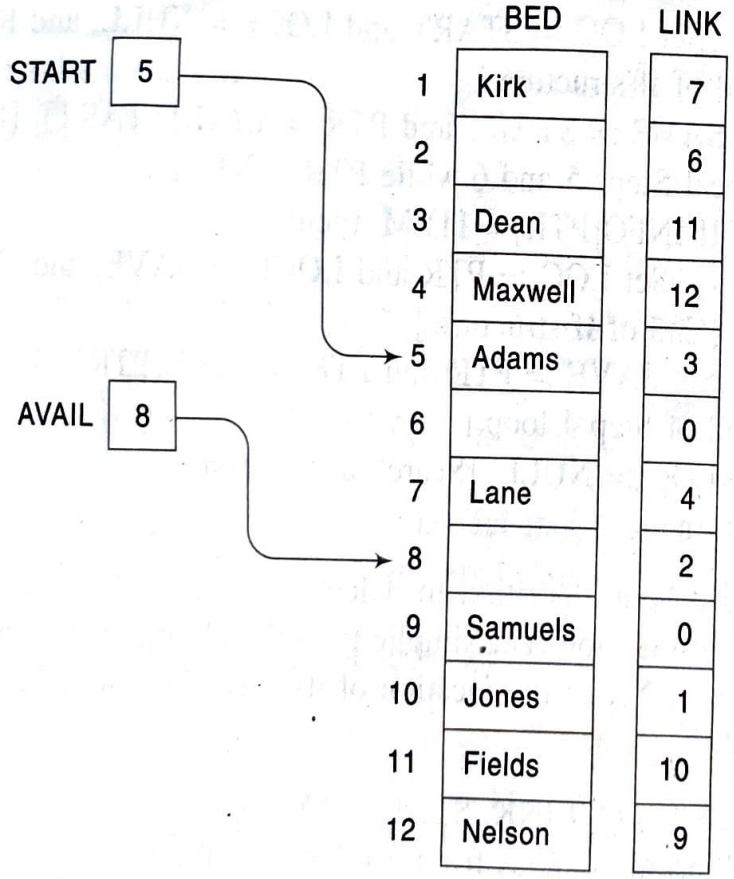


Fig. 5.28

(a) FINDB(BED, LINK, START, ITEM, LOC, LOCP)

1. Since START ≠ NULL, control is transferred to Step 2.
2. Since BED[5] = Adams ≠ Green, control is transferred to Step 3.
3. SAVE = 5 and PTR = LINK[5] = 3.
4. Steps 5 and 6 are repeated as follows:
   - **(a)** BED[3] = Dean ≠ Green, so SAVE = 3 and PTR = LINK[3] = 11.
   - **(b)** BED[11] = Fields ≠ Green, so SAVE = 11 and PTR = LINK[11] = 8.
   - **(c)** BED[8] = Green, so we have:
     LOC = PTR = 8 and LOCP = SAVE = 11, and Return.

(b) DELLOC(BED, LINK, START, AVAIL, ITEM)

1. Call FINDB(BED, LINK, START, ITEM, LOC, LOCP). [Hence LOC = 8 and LOCP = 11.]
2. Since LOC ≠ NULL, control is transferred to Step 3.
3. Since LOCP ≠ NULL, we have:
   LINK[11] = LINK[8] = 10.
4. LINK[8] = 2 and AVAIL = 8.
5. Exit.

Figure 5.28 shows the data structure after Green is removed from the patient list. We emphasize that only three pointers have been changed, LINK[11], LINK[8] and AVAIL.